Algorithm for insertion and Deletion in a queue (using pointers)

Implementing queue using pointers, the main drawback is that a node in the linked list occupies more memory space then a corresponding element in the array, since there all atleast two fields needed to store the information in a node, whereas in array representation only data field is there. However, the Mm space occupied by a node in linked list is not exactly twice the space used by a element of array representation. Moreover Linked representation (Dynamic method) makes an effective utilization of Mm. Another advantage of Linked representation became apparent when it is needed insert or delete element in the middle of a group of elements. This requires to shift one place all the elements so that a new slot can be made empty to insert new value. In array representation it is equivalent of doing a bulk of work with gaining no special result. Similar case for deletion to fill the gap due to deletion of element in array representation. Dynamic/Linked representation eleminates this stress/overhead to make process simple. In order words, the amount of work needed is independent of size of list.

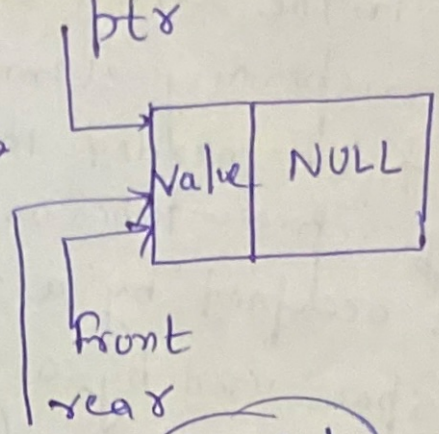
Addition of an new node b/w a queue in Linked representation requires creating a new node, inserting it on the required position by adjusting two pointers. whereas for deletion a node, only we require to adjust pointers and then free that node.

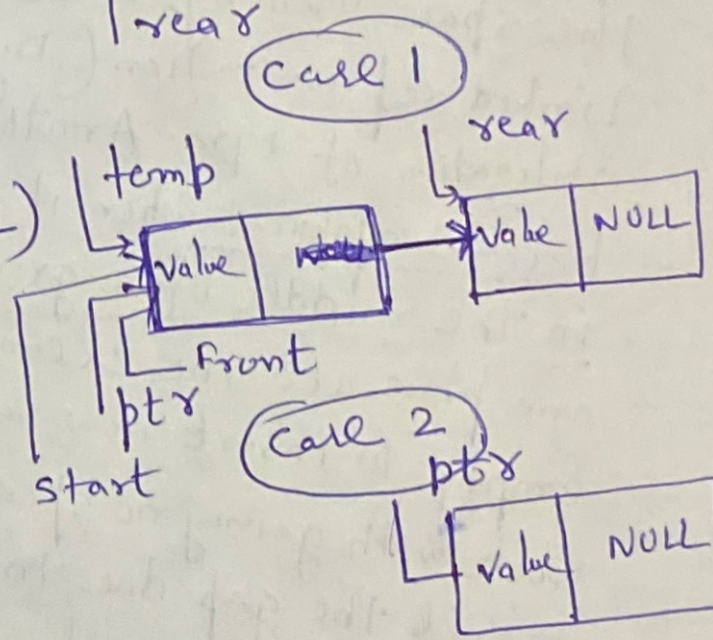
Let us see algorithm for adding a node in a queue:

```
struct queue-node
{
int n;
queue-node *next;
}*start = NULL;
```

Insertion Algorithm

1. Struct_queue *ptr, *temp)
2. temp = start [initialize temp]
3. ptr = new node [allocate Mm for new node]
4. ptr → ~~next~~ = value
5. ptr → next = NULL
6. IF (rear == NULL)
   set front = ptr;
   set rear = ptr;
   else
   while (temp → next != NULL)
   temp = temp → next
7. temp → next = ptr
   rear = ptr
8. end



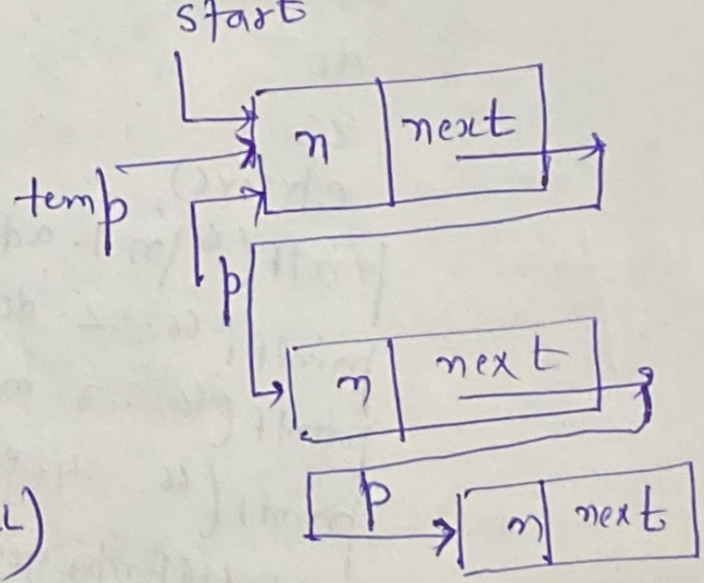


Deletion Algorithm

1. IF (front == NULL)
   write ("queue is empty and exit")
   else
   temp = start;
   value = temp → n;
   start = start → next
   free (temp)
   return (value)
2. exit / end.

```
#include<stdio.h>
#include<conio.h>
struct queue
  {
   int n;
 struct queue *next;
  } *start=NULL;
void add();
 int del();
void moue();
void main()
  {
  int a;
 char ch;
  do
  {
  clrscr();
printf("\n 1. add\n");
printf("...2. del\n");
printf("...3. move\n");
printf("...4. exit\n");
printf("Enter your choice\n");
 scanf("%d",&a);
switch(a)
  {
case 1: add();
    break;
case 2: printf("the deleted item is %d",del());
    break;
case 3: moue();
    break;
```

```
case 4: exit();
        break;
default : printf("wrong choice\n");
    }
    scanf("%c", &ch);
}
while(choice != 4);
}
void add()
{
struct queue *p, *temp;
 temp = start;
p = (struct queue *)malloc(sizeof(struct queue));
 printf("Enter the data\n");
  scanf("%d", &p->n);
  p->next = NULL;
if(start == NULL)
{
 start = p;
}
else
{
while(temp->next != NULL)
{
 temp = temp->next;
}
temp->next = p;
}
```



```
int del()
  {
  struct queue *temp;
    int item;
  if(start == NULL)
     {
      printf("queue is empty\n");
       getch();
       return(0);
     }
    else
     {
      temp = start;
       item = temp -> n;
       start = start -> next;
        free(temp);
     }
    return(item);
  }
void move()
   {
   struct queue *temp;
    temp = start;
while(temp -> next != NULL)
     {
      printf("%d", temp -> n);
       temp = temp -> next;
     }
   }
```